

LAB 14.2

Scenario

According to <https://www.fireeye.com/blog/threat-research/2018/06/bring-your-own-land-novel-red-teaming-technique.html>, one of most significant recent developments in sophisticated offensive operations is the use of "Living off the Land" (LotL) techniques by attackers. These techniques leverage legitimate tools present on the system, such as the PowerShell scripting language, in order to execute attacks. In response, defenders have developed detections for the malicious use of legitimate applications. These detections include suspicious parent/child process relationships, suspicious process command line arguments, and even deobfuscation of malicious PowerShell scripts through the use of Script Block Logging.

For this reason, attackers came up with an alternative to LotL techniques. Attacker's (and penetration testers) are now executing .NET assemblies entirely within memory. By developing custom C#-based assemblies, attackers no longer need to rely on the tools present on the target system; they can instead write and deliver their own tools, a technique fireeye calls Bring Your Own Land (BYOL).

A great example of BYOL is the *execute-assembly* command of CobaltStrike (<https://blog.cobaltstrike.com/2018/04/09/cobalt-strike-3-11-the-snake-that-eats-its-tail/>)

In this lab we will emulate such behavior and show you how you can proactively hook all .NET processes of a system and also perform an in-line hook of the *LoadImage* function (that is used under the hood to load the provided assembly.)

For more information please refer to <https://www.elastic.co/blog/hunting-memory-net-attacks>.

Goals

The learning objective of this lab is increase familiarity with .NET attacks by doing a hands-on analysis.

What you will learn

You will learn how to hook critical functions to proactively detect certain .NET-based threats on a host.

Recommended tools

- <https://github.com/endgameinc/ClrGuard>

Tasks

Task 1. Get familiar with ClrGuard

ClrGuard comes with some testing .bat files. One is benign and one is malicious. Try them out.

ClrGuard.exe is located at: *C:\Users\admin\Desktop\ClrGuard\dist*

Testing scripts are located at: *C:\Users\admin\Desktop\ClrGuard\Testing\scripts*

First, execute *ClrGuard.exe* and then run *fp.bat*. Notice what ClrGuard mentions.

Then, run *run.bat*. Notice what ClrGuard mentions.

Task 2. Emulate execute-assembly activity and use ClrGuard to proactively detect it

The easiest way in which we can cause the *LoadImage()* function to be called in order for a custom assembly to be loaded is through PowerShell.

PowerShell has the ability to load .NET assemblies from a specified location or even from a Byte Array (like attackers prefer). See an example below.

<https://powershell.one/tricks/assemblies/load-from-memory>

The above capability calls the native *LoadImage()* function under the hood to load the specified assembly, essentially emulating the *execute-assembly* command of Cobalt Strike.

Try to create such a code and see if ClrGuard detects it.

Hint: If you can't create your own PowerShell code, a *LoadNETAssembly.ps1* can be found in the Downloads directory of the machine, that performs what we have described above.

SOLUTIONS

Below, you can find solutions for each task. Remember though, that you can follow your own strategy, which may be different from the one explained in the following lab.

Task 1. Get familiar with ClrGuard

Execute *ClrGuard.exe* as administrator.

ClrGuard.exe is located at: *C:\Users\admin\Desktop\ClrGuard\dist*

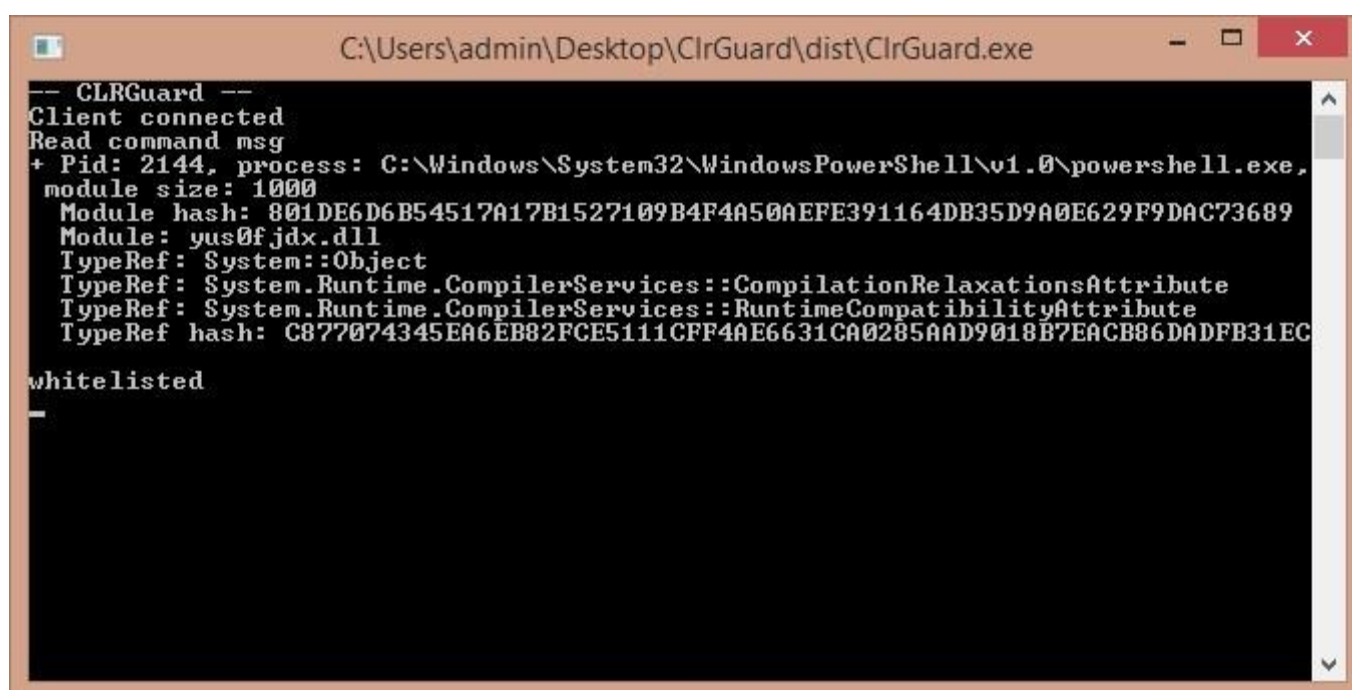
You should see the below.



Now, double-click *fp.bat*.

fp.bat is located at: *C:\Users\admin\Desktop\ClrGuard\Testing\scripts*

You should see the below.



The module being loaded is whitelisted by ClrGuard. Assembly loading can also be performed for legitimate purposes!

Now, execute the below.

Click the PowerShell icon that is pinned on the taskbar and execute the below.

```
powershell -ep bypass
cd C:\Users\admin\Desktop\ClrGuard\Testing\scripts
.\AddTypeRaceCondition.ps1
```

```
Windows PowerShell
Copyright (C) 2016 Microsoft Corporation. All rights reserved.

PS C:\Windows\System32\WindowsPowerShell\v1.0> powershell -ep bypass
Windows PowerShell
Copyright (C) 2016 Microsoft Corporation. All rights reserved.

PS C:\Windows\System32\WindowsPowerShell\v1.0> cd C:\Users\admin\Desktop\ClrGuard\Testing\scripts
PS C:\Users\admin\Desktop\ClrGuard\Testing\scripts> .\AddTypeRaceCondition.ps1
Unable to find type [Injected.Class].
At
C:\Users\admin\Desktop\ClrGuard\Testing\scripts\AddTypeRaceCondition.ps1:194
char:8
+ $ret = [Injected.Class]::ToString('Hello, from bypassed constrained l ...
+
+ CategoryInfo          : InvalidOperation: (Injected.Class:TypeName) [],
  RuntimeException
+ FullyQualifiedErrorId : TypeNotFound

PS C:\Users\admin\Desktop\ClrGuard\Testing\scripts> _
```

Note: If you receive the above error, Please re-run all the commands again.

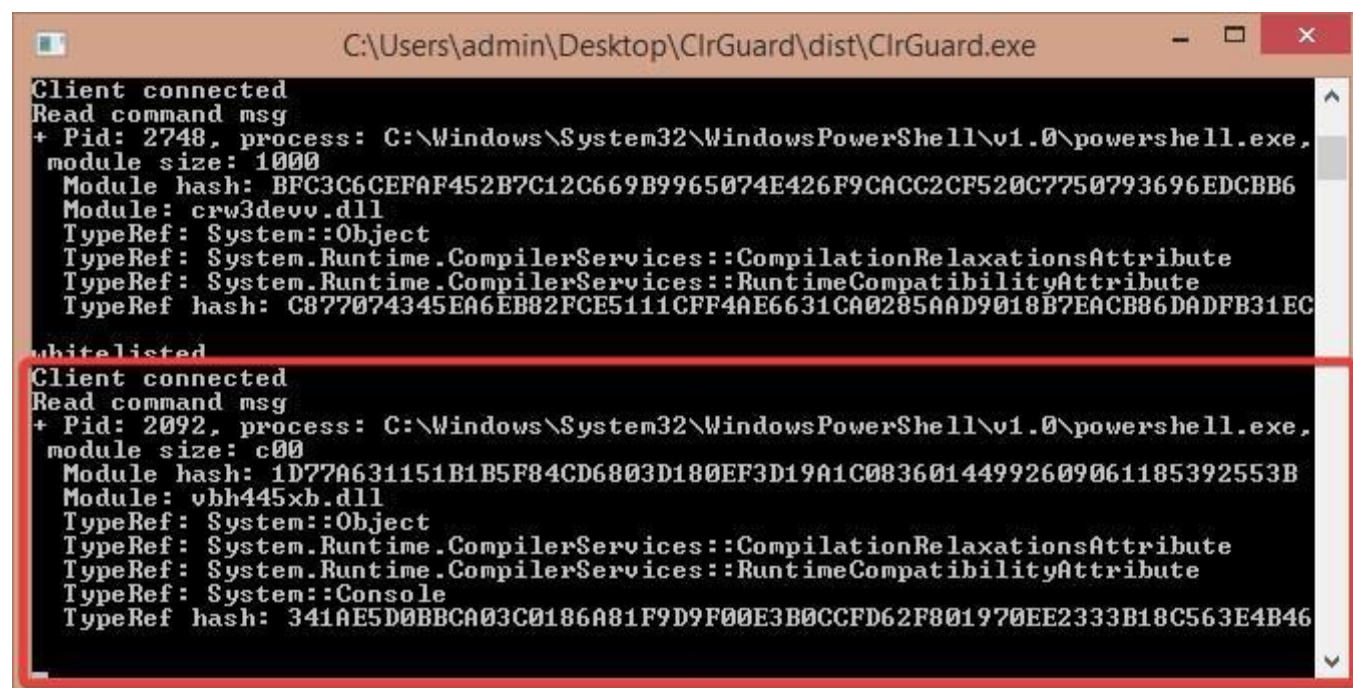
fp.bat is located at: *C:\Users\admin\Desktop\ClrGuard\Testing\scripts*

You should see the below.

```
PS C:\Users\admin\Desktop\ClrGuard\Testing\scripts> powershell -ep bypass
Windows PowerShell
Copyright (C) 2016 Microsoft Corporation. All rights reserved.

PS C:\Users\admin\Desktop\ClrGuard\Testing\scripts> .\AddTypeRaceCondition.ps1
***BLOCKED***
PS C:\Users\admin\Desktop\ClrGuard\Testing\scripts> _
```

Inside ClrGuard, you should see something similar to the below.



```
C:\Users\admin\Desktop\ClrGuard\dist\ClrGuard.exe
Client connected
Read command msg
+ Pid: 2748, process: C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe,
module size: 1000
Module hash: BFC3C6CEFAF452B7C12C669B9965074E426F9CACC2CF520C7750793696EDCBB6
Module: crw3devv.dll
TypeRef: System::Object
TypeRef: System.Runtime.CompilerServices::CompilationRelaxationsAttribute
TypeRef: System.Runtime.CompilerServices::RuntimeCompatibilityAttribute
TypeRef hash: C877074345EA6EB82FCE5111CF4AE6631CA0285AAD9018B7EACB86DADFB31EC
whitelisted
Client connected
Read command msg
+ Pid: 2092, process: C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe,
module size: c000
Module hash: 1D77A631151B1B5F84CD6803D180EF3D19A1C08360144992609061185392553B
Module: vhh445xb.dll
TypeRef: System::Object
TypeRef: System.Runtime.CompilerServices::CompilationRelaxationsAttribute
TypeRef: System.Runtime.CompilerServices::RuntimeCompatibilityAttribute
TypeRef: System::Console
TypeRef hash: 341AE5D0BBCA03C0186A81F9D9F00E3B0CCFD62F801970EE2333B18C563E4B46
```

The specific .net assembly loading was blocked by ClrGuard. Notice that there is no "whitelisted" string in the second entry.

Task 2. Emulate execute-assembly activity and use ClrGuard to proactively detect it

Let's now better emulate the execute-assembly command/functionality of Cobalt Strike.

In the *scripts* directory, there is a PowerShell script called *LoadNETAssembly.ps1*.

To execute it, perform the following.

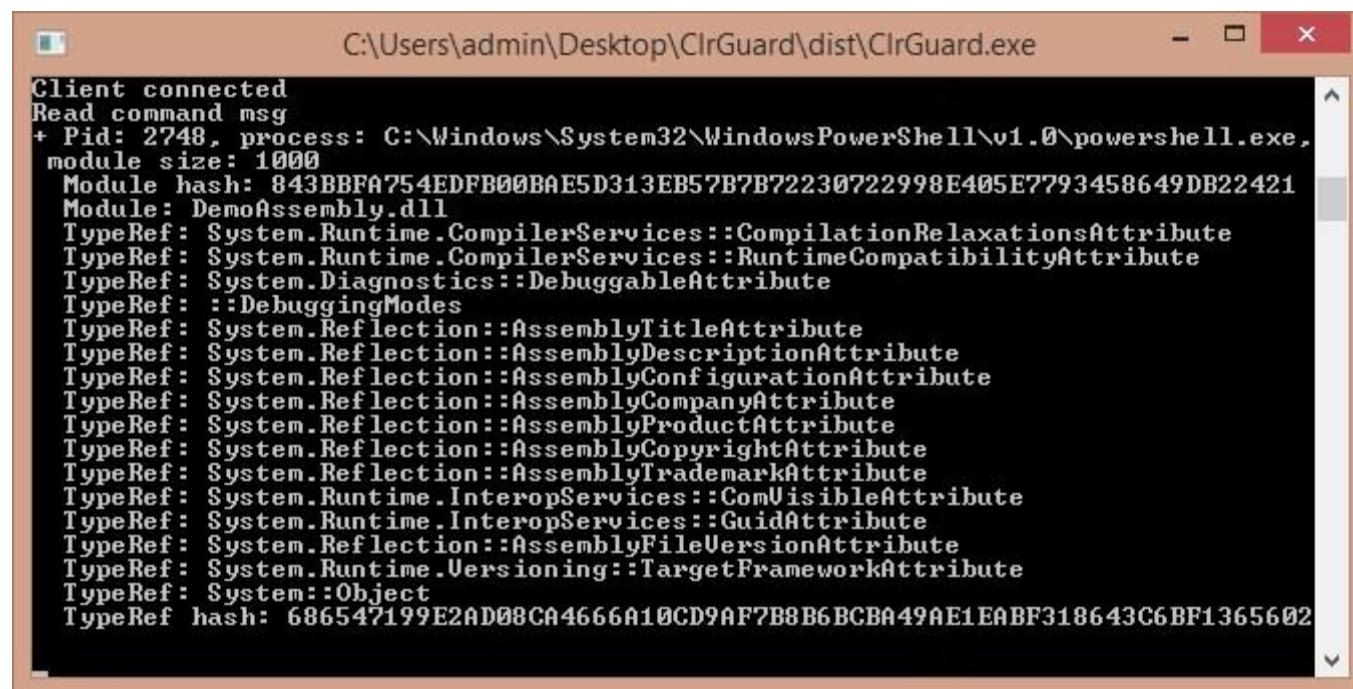
Click the PowerShell icon that is pinned on the taskbar and execute:

```
powershell -ep bypass
cd C:\Users\admin\Desktop\ClrGuard\Testing\scripts
.\LoadNETAssembly.ps1
```

You should see the below.

```
PS C:\Users\admin\Desktop\ClrGuard\Testing\scripts> .\LoadNETAssembly.ps1
***BLOCKED***
PS C:\Windows\System32\WindowsPowerShell\v1.0> _
```

Inside ClrGuard you should see something similar to the below.

A screenshot of a Windows command prompt window titled "C:\Users\admin\Desktop\ClrGuard\dist\ClrGuard.exe". The window shows the following output:

```
Client connected
Read command msg
+ Pid: 2748, process: C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe,
module size: 10000
Module hash: 843BBFA754EDFB00BAE5D313EB57B7B72230722998E405E7793458649DB22421
Module: DemoAssembly.dll
TypeRef: System.Runtime.CompilerServices::CompilationRelaxationsAttribute
TypeRef: System.Runtime.CompilerServices::RuntimeCompatibilityAttribute
TypeRef: System.Diagnostics::DebuggableAttribute
TypeRef: ::DebuggingModes
TypeRef: System.Reflection::AssemblyTitleAttribute
TypeRef: System.Reflection::AssemblyDescriptionAttribute
TypeRef: System.Reflection::AssemblyConfigurationAttribute
TypeRef: System.Reflection::AssemblyCompanyAttribute
TypeRef: System.Reflection::AssemblyProductAttribute
TypeRef: System.Reflection::AssemblyCopyrightAttribute
TypeRef: System.Reflection::AssemblyTrademarkAttribute
TypeRef: System.Runtime.InteropServices::ComVisibleAttribute
TypeRef: System.Runtime.InteropServices::GuidAttribute
TypeRef: System.Reflection::AssemblyFileVersionAttribute
TypeRef: System.Runtime.Versioning::TargetFrameworkAttribute
TypeRef: System::Object
TypeRef hash: 686547199E2AD08CA4666A10CD9AF7B8B6BCBA49AE1EABF318643C6BF1365602
```

Notice that there is no "whitelisted" string in the last entry.

Note that there are *execute-assembly* variations included in multiple attacking frameworks. The detection method covered doesn't work only against Cobalt Strike.